```
30 def your_number do
 -   twilio_config = Application.get_env(:phone_app, :twilio, [])
 -   Keyword.fetch!(twilio_config, :number)
 - end
```

At this point, all of these lines of code should feel right at home to you.

Our function starts on line 2 by defining the parameters needed to deliver an SMS message. We invoke our SMS API context function on line 8, which sends a message over HTTP. Finally, we handle success on line 20 by persisting our SMS message to the database. And if we get errors along the way, we return those to the caller.

We're almost done with the code for this section. Let's make one more adjustment, and then you can try out your app!

See Your App in Action

An SMS app needs two parties to make a conversation. Because this is a mock API, you'll have to talk to yourself. The MockServer app includes a way to reply to messages, but it requires a new route to be created. Let's add that and then use it from end to end.

Create the Webhook Controller

Create lib/phone_app_web/controllers/webhook/twilio_controller.ex and add the following code:

phone_app/lib/phone_app_web/controllers/webhook/twilio_controller.ex

```
defmodule PhoneAppWeb.Webhook.TwilioController do
  use PhoneAppWeb, :controller

  def sms(conn, params) do
    persist_message(params)

    conn
    |> put_resp_content_type("text/xml")
    |> send_resp(200, incoming_sms_response())
  end

  defp persist_message(params) do
    message = %{
      message_sid: params["MessageSid"],
      account_sid: params["AccountSid"],
      body: params["Body"],
      from: params["From"],
      to: params["To"],
      status: params["SmsStatus"],
      direction: :incoming
    }
```